

Failure (a loop that won't converge, and a cost-cutting pass) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `convergence_and_cost_cutting_practice.py` : two production risks in one file: a revision loop that never agrees, and a pipeline that costs too much. Both are proven here with a hard limit and a measured cut, before Project 4 depends on either. **If not:** Project 4's Writer/Reviewer pair could loop without a clear report, and a "cheaper" prompt could quietly break answers.

This exercise has two independent parts — read both. Read both depths of each — they're not "wrong, right," they're 2 real, valid ways to solve the same problem, with real tradeoffs between them.

Basic Version

Approach 1 — the direct way, Part 1: a loop that hits its limit

Story: give the critic a bar nothing can pass, on purpose, and confirm the loop stops and says so. **If not:** the first time you saw a non-converging loop would be in production, running up a bill.

```
# convergence_and_cost_cutting_practice.py
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.types import Command

MAX_REVISIONS = 3

class LoopState(TypedDict):
    draft: str
    revision_count: int
    converged: bool

def writer(state):
    draft = f"[draft attempt {state['revision_count'] + 1}]"
    return Command(update={"draft": draft}, goto="critic")
```

```
def critic(state):
    passed = False # criteria are impossible on purpose
    if passed:
        return Command(update={"converged": True}, goto=END)

    new_count = state["revision_count"] + 1
    # why: the hard limit – the loop must end, agreed or not
    if new_count >= MAX_REVISIONS:
        update = {"revision_count": new_count, "converged": False}
        return Command(update=update, goto=END)
    return Command(update={"revision_count": new_count}, goto="writer")

builder = StateGraph(LoopState)
builder.add_node("writer", writer)
builder.add_node("critic", critic)
builder.add_edge(START, "writer")
graph = builder.compile()

result = graph.invoke({"draft": "", "revision_count": 0, "converged": False})
if result["converged"]:
    print("Accepted:", result["draft"])
else:
    print(f"Couldn't converge after {result['revision_count']} attempts.")
```

Expected output:

```
Couldn't converge after 3 attempts.
```

Approach 1 — Part 2: find the expensive stage, cut it, remeasure

Story: before cutting cost, measure it — reusing `supervisor_compare` 's own report, so the numbers match what you already trust. **If not:** you'd cut whichever stage *feels* expensive.

```
# convergence_and_cost_cutting_practice.py
# how: reuse the measured pieces from the architecture comparison file
from architecture_comparison_practice import run_supervisor

def profile_pipeline(topic: str) -> dict:
    return run_supervisor(topic) # from supervisor_compare, reused unchanged

report = profile_pipeline("climate change")
```

```
print("tokens by stage would need per-stage breakdown -- see")
print("Intermediate for that")
print("total:", report["total_tokens"], "tokens,")
print(report["total_seconds"], "seconds")
```

Both work: the loop correctly stops instead of running forever, and the pipeline can be measured.

Part 1 is missing any record of *why* the critic rejected each time. Part 2 is missing a per-stage breakdown (it only has a total) and any actual cut — both fine for a first pass that confirms the basic mechanics.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — Part 1: a real rejection reason on every attempt

Story: "gave up" alone tells nobody what to fix; the critic's reason does. **If not:** every failed run would need to be re-run by hand just to find out why it failed.

```
# convergence_and_cost_cutting_practice.py
class LoopState(TypedDict):
    draft: str
    revision_count: int
    converged: bool
    last_rejection_reason: str

def writer(state: LoopState) -> Command:
    draft = f"[draft attempt {state['revision_count'] + 1}]"
    return Command(update={"draft": draft}, goto="critic")

def critic(state: LoopState) -> Command:
    # impossible criteria on purpose: "under 10 words AND cites 3 sources"
    # for a placeholder draft -- nothing generated here could ever pass
    passed = False
    reason = "draft must be under 10 words and cite 3 sources; neither was met"

    if passed:
        return Command(update={"converged": True}, goto=END)

    new_count = state["revision_count"] + 1
    if new_count >= MAX_REVISIONS:
        return Command(
            update={
                "revision_count": new_count,
                "converged": False,
                "last_rejection_reason": reason,
            },
            goto=END,
        )
    return Command(
        update={"revision_count": new_count, "last_rejection_reason": reason},
```

```

        goto="writer",
    )

builder = StateGraph(LoopState)
builder.add_node("writer", writer)
builder.add_node("critic", critic)
builder.add_edge(START, "writer")
graph = builder.compile()

initial_state = {
    "draft": "",
    "revision_count": 0,
    "converged": False,
    "last_rejection_reason": "",
}
result = graph.invoke(initial_state)
if result["converged"]:
    print("Accepted:", result["draft"])
else:
    attempts = result["revision_count"]
    reason = result["last_rejection_reason"]
    print(f"Couldn't converge after {attempts} attempts: {reason}")

```

Expected output (shown wrapped onto 2 lines just to fit the page — really one line of output):

```

Couldn't converge after 3 attempts: draft must be under 10 words and
cite 3 sources; neither was met

```

Approach 1 — Part 2: per-stage profiling, then one real cut

Story: a total can't tell you *where* to cut; per-stage numbers can, and one targeted cut can then be measured on its own. **If not:** you'd change several things at once and never know which one saved the money.

```

# convergence_and_cost_cutting_practice.py
import time

from architecture_comparison_practice import model, research, write
from architecture_comparison_practice import StageResult

def profile_stages(topic: str) -> dict:
    research_result = research(topic)
    write_result = write(research_result.text)

```

```

    return {"research": research_result, "write": write_result}

def find_most_expensive(profile: dict) -> str:
    most_expensive_stage = None
    highest_tokens = -1
    for stage, result in profile.items():
        if result.tokens > highest_tokens:
            highest_tokens = result.tokens
            most_expensive_stage = stage
    return most_expensive_stage

profile = profile_stages("climate change")
for stage, result in profile.items():
    print(f"{stage}: {result.tokens} tokens, {result.seconds:.2f}s")

expensive_stage = find_most_expensive(profile)
print(f"most expensive stage: {expensive_stage}")

```

Expected output (exact numbers vary by run):

```

research: 210 tokens, 1.84s
write: 96 tokens, 0.91s
most expensive stage: research

```

```

# convergence_and_cost_cutting_practice.py
def research_shorter_prompt(topic: str) -> StageResult:
    start = time.perf_counter()
    prompt = f"List the top 3 causes of {topic}." # shorter than the original
    response = model.invoke(prompt)
    elapsed = time.perf_counter() - start
    call_tokens = response.usage_metadata["total_tokens"]
    return StageResult(response.content, call_tokens, elapsed)

before = research("climate change")
after = research_shorter_prompt("climate change")
saved_pct = (before.tokens - after.tokens) / before.tokens * 100
print(f"before: {before.tokens}t, after: {after.tokens}t")
print(f"saved: {saved_pct:.0f}%")

```

Expected output (exact numbers vary by run):

before: 210t, after: 152t
saved: 28%

Difference from Basic: Part 1's critic gives a specific, real reason instead of nothing — "under 10 words and cites 3 sources" is a concrete, genuinely unsatisfiable bar rather than a bare `passed = False`, and the final report explains *why* it failed, not just that it did. Part 2 now has a real per-stage breakdown (not just a total) that correctly identifies which stage to target, plus one actual cut (a shorter prompt) with a specific measured saving. Neither part yet proves the cut is safe on more than a single manual check, or shows whether the critic's rejection reason ever changes across attempts — that's what Approach 2 adds.

Approach 2 — Part 1: a full rejection history, diagnosing the failure

Story: the same reason three times means the criteria are impossible; three different reasons means they're inconsistent — only the full history can tell those apart. **If not:** you'd rewrite the Writer when the Critic was the problem.

```
# convergence_and_cost_cutting_practice.py
class LoopState(TypedDict):
    draft: str
    revision_count: int
    converged: bool
    rejection_history: list

def critic(state: LoopState) -> Command:
    passed = False
    reason = "draft must be under 10 words and cite 3 sources; neither was met"
    history = state["rejection_history"] + [reason]

    if passed:
        return Command(update={"converged": True}, goto=END)

    new_count = state["revision_count"] + 1
    if new_count >= MAX_REVISIONS:
        return Command(
            update={
                "revision_count": new_count,
                "converged": False,
                "rejection_history": history,
            },
            goto=END,
        )
    return Command(
        update={"revision_count": new_count, "rejection_history": history},
```

```

        goto="writer",
    )

def writer(state: LoopState) -> Command:
    draft = f"[draft attempt {state['revision_count'] + 1}]"
    return Command(update={"draft": draft}, goto="critic")

# why: rebuild the graph – it must use THIS LoopState and critic,
# or rejection_history would never be saved
builder = StateGraph(LoopState)
builder.add_node("writer", writer)
builder.add_node("critic", critic)
builder.add_edge(START, "writer")
graph = builder.compile()

def build_report(result: dict) -> str:
    if result["converged"]:
        return f"Accepted: {result['draft']}"

    reasons = result["rejection_history"]
    unique_reasons = set(reasons)
    if len(unique_reasons) == 1:
        pattern = "the same reason every time -- criteria are likely impossible"
    else:
        pattern = "a different reason each time -- criteria may be inconsistent"

    attempts = result["revision_count"]
    lines = [f"Couldn't converge after {attempts} attempts ({pattern}):"]
    for i, reason in enumerate(reasons, start=1):
        lines.append(f"  attempt {i}: {reason}")
    return "\n".join(lines)

initial_state = {
    "draft": "",
    "revision_count": 0,
    "converged": False,
    "rejection_history": [],
}
result = graph.invoke(initial_state)
print(build_report(result))

```

Expected output (first line shown wrapped onto 2 lines just to fit the page — really one line of output):

```

Couldn't converge after 3 attempts (the same reason every time -- criteria
are likely impossible):
  attempt 1: draft must be under 10 words and cite 3 sources; neither was met
  attempt 2: draft must be under 10 words and cite 3 sources; neither was met
  attempt 3: draft must be under 10 words and cite 3 sources; neither was met

```

This is the report a reader can actually act on: the repeated identical reason tells them the criteria themselves are the problem, not the writer — a different scenario entirely from a report that just said "gave up after 3 tries."

Approach 2 — Part 2: proving the cut against a real test set

Story: a saving on one input might have broken the answer on another; five inputs, compared before and after, show both the saving and any damage. **If not:** you'd ship a cheaper prompt that silently got worse.

```

# convergence_and_cost_cutting_practice.py
def compare_before_after(test_inputs: list[str]) -> None:
    before_total_tokens = 0
    after_total_tokens = 0
    mismatches = []

    for topic in test_inputs:
        before = research(topic)
        after = research_shorter_prompt(topic)
        before_total_tokens += before.tokens
        after_total_tokens += after.tokens

        # crude but honest quality check: did the shorter prompt still
        # produce a non-trivial answer, roughly the same length ballpark?
        length_ratio = len(after.text) / max(len(before.text), 1)
        if length_ratio < 0.5:
            mismatches.append((topic, length_ratio))

    tokens_saved = before_total_tokens - after_total_tokens
    saved_pct = tokens_saved / before_total_tokens * 100
    print(f"tested {len(test_inputs)} inputs")
    print(f"tokens before: {before_total_tokens}, after: {after_total_tokens}")
    print(f"saved: {saved_pct:.0f}%")
    if mismatches:
        print(f"POSSIBLE QUALITY DROP on {len(mismatches)} input(s):")
        print(mismatches)
    else:
        print("no output-length red flags on any test input")

```

```
test_inputs = [
    "climate change",
    "inflation",
    "sleep quality",
    "urban housing costs",
    "renewable energy adoption",
]
compare_before_after(test_inputs)
```

Expected output (exact numbers vary by run):

```
tested 5 inputs
tokens before: 1046, after: 758
saved: 28%
no output-length red flags on any test input
```

28% is close to, but short of, the 30% target — the honest next step, not shown as code here, is trying a second lever (a cheaper model for this stage, on top of the shorter prompt) and re-running this exact same comparison, rather than declaring victory on a number that's close but not there yet.

Difference between Approach 1 and Approach 2: Approach 1's Part 1 gives one real reason but throws it away after each attempt, so the final report can't say whether the failure pattern was consistent or not. Approach 1's Part 2 measures one cut on one input and calls it done. Approach 2 turns both single manual checks into systematic ones — Part 1's full history turns "it failed" into "it failed *this specific, diagnosable way*," and Part 2's 5-input before/after comparison turns "it's cheaper" into "it's cheaper, and here's the proof nothing broke, or here's exactly which input needs a closer look."

Which one should you actually write? For any revision loop with a real hard limit — which every generator ↔ critic pattern needs, per this document's Core Concepts — Approach 2's full rejection history is worth keeping, since the cost of storing a list of strings is negligible next to the cost of an on-call engineer re-running a failed pipeline from scratch just to find out why it failed. For a cost cut, never ship one you've only checked against a single example — Approach 2's fixed test-set comparison is the minimum bar, and it's the exact same discipline `sequential_measure` and `supervisor_compare` already trained: don't trust a number you haven't measured against a real, repeated test set. If you're short on time, Approach 1's single-input check is acceptable for a quick local experiment, but re-run the full 5-input comparison before anything reaches Project 4's real pipeline.